Gossip Out of Contact

Coverage and Task Allocation in Communication-Restricted Environments, after Bramblett and Bezzo, Recreated, with the Gossip as an Epistemic Planning Task on ePlanSys

Haniel Ulises

Epistemic Robotics

23 October 2026

Bramblett and Bezzo [1] let a team of robots disconnect to cover an unknown area faster. Every robot propagates the same set of particles, the common belief, from what all last shared, and tracks its own, so that it stays where the others believe it to be. A robot that finds a task needing several robots allocates it by auctions and tells the robots it needs by going to their particles. They write the epistemic state in dynamic epistemic logic and solve the allocation with heuristics. This report recreates the paper in three parts: its simulation study, in a kinematic simulator that reproduces the comparison of its Figures 6 and 8 (720 runs); the decision at a task found, written as an EPDDL planning task whose policies Aletheia computes and ePlanSys executes; and its laboratory experiments, with three TurtleBot3 robots in Gazebo (96 runs). A formal analysis states the model, proves the paper's update rules as properties of the product update, shows that the literal reading of its rule for disconnected robots breaks the equivalence of knowledge and belief it assumes, proves the planning task solvable under the assumption that a busy robot comes back, and gives the counterexample, from the paper's own task mix, in which two policies wait for each other. In the kinematic study the paper's ordering reappears, with gaps of 8.1 and 34.4 s in Case II against its 16 and 35; the policies, all 320 found in at most 0.31 s, are at parity with the procedure where tasks compete and slower where everyone must be told. In Gazebo 96 of 96 runs ended with every task done. Every canonical policy satisfies the paper's update rules in every world and can be followed by every robot on what it observes; under the literal reading of the paper's rule for disconnected robots, a task that needs news to pass through a robot, or a wait for a busy one, has no policy with consistent beliefs at all.

Contents

1 Scope **2**

2 The paper **2**

3 The recreation **3**

4 The planning task **3**
4.1 Models, events and the product update . . . 3
4.2 The instance . . . 3
4.3 The actions . . . 4
4.4 A policy . . . 4

5 Analysis **5**
5.1 The paper's update rules . . . 5
5.2 The literal reading of eq. (18) . . . 6
5.3 Joint policies . . . 7
5.4 Solvability . . . 7
5.5 The second ruleset rests on belief . . . 8
5.6 When exclusion happens . . . 8
5.7 Size and planning cost . . . 8

6 The paper's simulation study **9**

7 The laboratory experiments in Gazebo **10**

8 Implementation defects **12**

9 Limitations **13**

1 Scope

This report accompanies the page *Gossip out of contact* of the Epistemic Robotics project, the EPDDL workspace `gossip-coverage` and the package `gossip_coverage_demo`. Section 2 summarises the paper. Section 3 says what each of its constructs is here. Section 4 states the planning task formally and Section 5 proves what holds of it and what does not. Sections 6 and 7 report the kinematic study and the Gazebo runs, Section 8 the defects found on the way, and Section 9 the limits of what is claimed.

2 The paper

Problems. Problem 1 of [1] asks for a distributed policy by which N_r robots cover a partly unknown area with intermittent communication; Problem 2 for a distributed policy that allocates tasks found during the coverage when an allocation may need robots out of contact. A task τ has a place, a number N_τ of robots it needs at once, and a reward; it is done when N_τ robots are within a radius of it.

Belief and empathy. Robot i holds particles $p^i_{j,k}$: its belief about robot j's belief about robot k's state. The empathy particles $p^i_{j,i}$ are what the others believe of i. When all robots are in contact they share their states, maps and tasks, and from then on each propagates the same particles, the common belief set C^i, by the same coverage, until the next moment all are in contact (eq. (14)). A robot out of contact tracks its own common-belief particle, so that it is where the others believe it to be.

Coverage. Each common-belief particle claims the frontier points of its Voronoi cell (eq. (9)); the utility of a point is its cost plus a penalty Δ outside the cell (eq. (10)); a particle goes to the point of least utility (eqs. (11) and (12)) by A* and an artificial potential field.

Updates and rules. The action library is $\{\mathit{perceive}(\varphi), \mathit{announce}(\varphi)\}$ and $K_i\,\mathit{present}(\tau)$ reads "i knows the place, need and value of τ". An announcement reaches the robots in contact, and "disconnected robots' knowledge is not impacted" (eq. (18)). Two rulesets govern the planning: a robot sought at its particle and not found is excluded from the allocation, as busy; and a robot that finds a task it believes a teammate found first, because the teammate's coverage passed it, leaves it to that teammate until they are in contact. The policy is a joint one: "robots need to map indistinguishable epistemic states to the same actions".

Allocation. A task found is allocated by a sequential single-item auction (Algorithm 1), in which a robot in contact bids for each robot out of contact from its belief; the bundles are reordered for least makespan (Algorithm 2); and a gossip auction (Algorithm 3) assigns who tells whom, a robot told joining the tellers.

Evaluation. Fifteen random 50×50 m areas with 10 to 20 unknown 1 m obstacles, teams of 2, 3, 5 and 8, robots of 3 m/s with a 5 m lidar. Case I: one task, known to exist, needing an unknown number of robots; Case II: an unknown number of tasks needing in all no more robots than the team has. The baselines are the flock, which keeps every robot within 10 m, and the ideal method, with communication across the area. The paper reports, in Case I, that its method is 11 s slower than the ideal one and 13 s faster than the flock on average, with standard deviations of 11, 14 and 48 s for its method, the ideal one and the flock, the flock being better with two robots; in Case II, 16 s slower than the ideal method and 35 s faster than the flock. Its laboratory experiments run two or three ROSbots in a 4×5.5 m space with a sensing and communication range of 1 m: a two-robot task with two and with three robots (Figs. 9 and 10), and a two-robot and a three-robot task with three robots (Fig. 11).

3 The recreation

In the paper	Here
Common belief set C^i, propagated by the coverage	`gossip/belief.py`: one particle per robot, propagated from the state all last shared, reset when all are in contact and new knowledge is shared
Empathy: a robot tracks its particle	every robot tracks its own particle; in Gazebo within 0.18 m in median and 0.9 m nine times in ten
Coverage, eqs. (9)–(12)	Voronoi ownership of frontier points, utility with Δ, a turn term and a gain term, A* and a potential field
Algorithms 1–3, rulesets 1 and 2	`gossip/alloc.py` and `gossip/sim.py`, the *procedure*
World $w = \{\sigma_i\}$, statuses	atoms: at the particle, `out`/`by` (gossiping), `sited` (completing a goal), `busy` (other work)
perceive, *announce*	the event model call: met (the announcement between the two in contact) or absent (the perception of the empty particle)
K_i *present*(τ)	K_i *task*, a precondition of telling and of going to the site
Joint policy	a policy in which every action branches on what its actor observes (Aletheia's `--subjective auto`), checked by `tools/check_policy.py`
Planning task $\Pi = (s_t^i, A, \gamma)$	the EPDDL problem written by the finder from what it knows (`gossip/epddl.py`)
MATLAB simulations	the kinematic simulator, `tools/study.py`
ROSbot experiments	three Burgers in Gazebo, ePlanSys executing the policy

Table 1: The paper's constructs and what they are in the recreation.

Four formulations run on the same robots. The *procedure* is the paper's framework as written, with one strengthening: a robot excluded by the first ruleset is admitted again after 60 s, and a robot that holds a task it cannot allocate tries again every 5 s, so that an exclusion delays a task and does not lose it. The *ideal* method is the procedure with communication across the area; the *flock* keeps every robot within the radio range of every other. The *epistemic* formulation is the procedure with the decision at a task found made by a policy of the EPDDL task of Section 4: in the kinematic simulator by a port of ePlanSys's scheduler (`gossip/schedule.py`), and in Gazebo by ePlanSys itself. The epistemic formulation does not apply the second ruleset (Proposition 8).

4 The planning task

4.1 Models, events and the product update

Definition 1 (Epistemic model)**.** Over agents $\mathcal{A}$ and atoms P, an epistemic model is $\mathcal{M} = (W, (R_a)_{a\in\mathcal{A}}, V, W_d)$ with W a finite set of worlds, $R_a \subseteq W \times W$ an equivalence for each agent, $V : W \to 2^P$ and $W_d \subseteq W$ the designated worlds. $\mathcal{M}, w \models K_a\varphi$ when $\mathcal{M}, v \models \varphi$ for every v with wR_av; $Kw_a\varphi$ abbreviates $K_a\varphi \vee K_a\neg\varphi$.

Definition 2 (Event model and product update [4, 3])**.** An event model is $\mathcal{E} = (E, (Q_a), pre, post, E_d)$. Here the relation of an agent depends on its observability class at the world where the action happens: a *Fully* observer has $Q(e) = \{e\}$, an *Unsure* one $Q(e) = E$, and an *oblivious* one $Q(e) = \{e_{nil}\}$, where the null event has $pre = \top$ and no effects. The product $\mathcal{M} \otimes \mathcal{E}$ has worlds $\{(w, e) : \mathcal{M}, w \models pre(e)\}$, $(w, e)\, R'_a\, (v, f)$ iff wR_av and $f \in Q_{a,w}(e)$, valuations changed by $post(e)$, and designated worlds $\{(w, e) : w \in W_d, e \in E_d\}$.

4.2 The instance

Definition 3 (Instance)**.** An instance is $I = (\mathcal{A}, f, k, c, K, B, b, N)$: the robots $\mathcal{A}$, the finder f, the need $k \leq |\mathcal{A}|$, the case $c \in \{1, 2\}$, the robots $K \ni f$ in contact with the finder, the robots

$B \subseteq \mathcal{A} \setminus K$ that may be busy, a bound $b \leq |B|$ on how many are, and the neighbour relation $N \subseteq \mathcal{A}^2$: the pairs whose cells in the Voronoi partition of the believed particle positions share a border (the Delaunay edges; all pairs for three robots or fewer).

The atoms are *task*, *done*, and for robots a, a': $busy_a$, out_a, $sited_a$, $by_{aa'}$, $called_{aa'}$; the counter $c_0, \ldots, c_8$; and the rigid facts $need_k$ and $near_{aa'}$ for $(a, a') \in N$.

Definition 4 (Initial model)**.** $\mathcal{M}_0(I)$ has a world $w_{S,t}$ for every $S \subseteq B$ with $|S| \leq b$ and $t \in \{0, 1\}$. In $w_{S,t}$, $busy_a$ holds iff $a \in S$, *task* iff $t = 1$, and c_0; no other atom holds. $W_d = \{w_{S,1}\}$, and $w_{S,t} \, R_a \, w_{S',t'}$ iff $(a \in B \Rightarrow (a \in S \Leftrightarrow a \in S'))$ and $(a \in K \Rightarrow t = t')$.

This is the model plank builds from the problem's finitary S5 theory: $C(\neg busy_a)$ for $a \notin B$, $C(Kw_a \, busy_a)$ for $a \in B$, $C(Kw_a \, task)$ for $a \in K$, $C(\neg Kw_a \, task)$ otherwise, and the bound. Its size is the first fact of the analysis:

Lemma 1. $|W_d| = \sum_{j=0}^{b} \binom{|B|}{j}$ and $|W| = 2|W_d|$.

For the canonical instances this gives 1 and 2 (no robot may be busy), 4 and 8 (three robots, two may be busy) and 29 and 58 (eight robots, seven may be busy, at most two), the counts plank reports.

4.3 The actions

Definition 5 (Call)**.** $\mathsf{call}(i, j)$ has events e_{met}, e_{abs} and e_{nil}, the first two designated, with

$$pre(e_{met}) = near_{ij} \wedge free_i \wedge K_i task \wedge \neg busy_j \wedge \neg out_j \wedge \neg sited_j \wedge task, \quad post(e_{met}) = \{called_{ij}\},$$
$$pre(e_{abs}) = near_{ij} \wedge free_i \wedge K_i task \wedge busy_j, \qquad post(e_{abs}) = \{out_i, by_{ij}\},$$

where $free_i = \neg out_i \wedge \neg sited_i \wedge \neg busy_i$. The caller i is Fully; j is Fully at worlds where $\neg busy_j$ and Unsure where $busy_j$; every other robot is Unsure.

$\mathsf{await}(i, j)$ has one designated event, with precondition $by_{ij} \wedge busy_j \wedge task \wedge K_i \, busy_j$ and effects $\neg busy_j$, $\neg by_{ij}$, $\neg out_i$, $called_{ij}$; i and j are Fully. $\mathsf{leave}(i, j)$ undoes by_{ij} and out_i. $\mathsf{go\text{-}site}_n(r)$ requires c_n, $\neg need_{n+1}$, $free_r$, $\neg sited_r$ and $K_r task$, and moves the counter to c_{n+1}; $\mathsf{do\text{-}task}_{n+1}(r)$ is the same arrival under $need_{n+1}$, and adds *done*. Their actor is Fully and every other robot Unsure. The goal is *done* in Case II and $done \wedge \bigwedge_{a \in \mathcal{A}} K_a task$ in Case I, where every robot must learn of the task.

4.4 A policy

Figure 1 is the policy Aletheia returns for `hardware_case2`, the paper's Fig. 11 at the moment its three-robot task is found: r_1 knows of the task, and r_2 and r_3 may be busy. It calls r_2; met, it sets out for the site while r_2 calls r_3; absent, it waits at r_2's particle, which it may, since it knows r_2 busy. Every branch is taken by the robot whose call it follows.

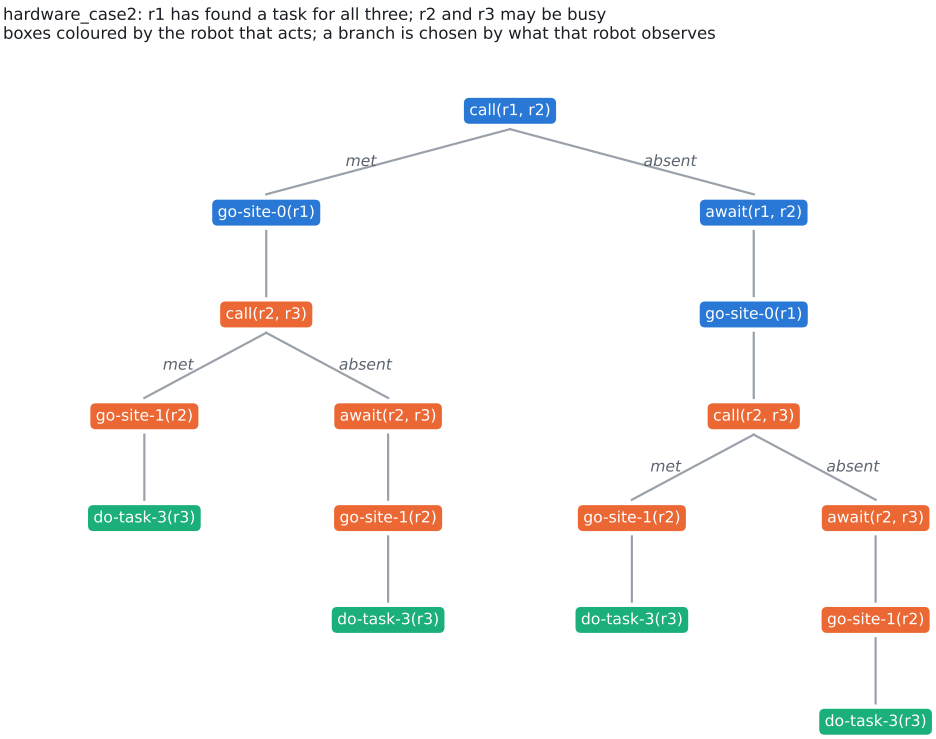


Figure 1: The policy of `hardware_case2`: depth 7 in its worst world, four designated worlds.

5 Analysis

5.1 The paper's update rules

Proposition 1 (Announcement in contact; eq. (18))**.** If $\mathcal{M}, w \models pre(e_{met})$ for $\mathsf{call}(i, j)$, then $\mathcal{M} \otimes \mathcal{E}, (w, e_{met}) \models K_j task \wedge K_i K_j task \wedge K_j K_i task$.

Proof. At w, $\neg busy_j$, so j is Fully and $R'_j(w, e_{met}) = \{(v, e_{met}) : wR_jv,\ \mathcal{M}, v \models pre(e_{met})\}$. $task$ is a conjunct of $pre(e_{met})$ and no event changes it, so every pair in $R'_j(w, e_{met})$ satisfies $task$, and $K_j task$ holds. The caller is Fully everywhere, so $R'_i(w, e_{met})$ consists of pairs (v, e_{met}) with $\mathcal{M}, v \models pre(e_{met})$; at each, $\neg busy_j$ holds at v, so j is Fully there and the same argument gives $K_j task$ at (v, e_{met}); hence $K_i K_j task$. For $K_j K_i task$: at every $(v, e_{met}) \in R'_j(w, e_{met})$ the pairs i considers are again met pairs, which satisfy $task$. □

Proposition 2 (Robots out of contact are not affected; eq. (18))**.** Let $k \notin \{i, j\}$, Unsure at every world, and $e \in \{e_{met}, e_{abs}\}$ with $\mathcal{M}, w \models pre(e)$. Then (a) $\mathcal{M} \otimes \mathcal{E}, (w, e) \models K_k task$ iff $\mathcal{M}, w \models K_k task$; (b) if $\mathcal{M}, w \not\models K_k K_j task$ then $\mathcal{M} \otimes \mathcal{E}, (w, e) \not\models K_k K_j task$; (c) if $\mathcal{M}, w \not\models Kw_k\ busy_j$ then $\mathcal{M} \otimes \mathcal{E}, (w, e) \not\models Kw_k\ busy_j$; and (d) the actual pair is among those k considers possible.

Proof. Since k is Unsure, $R'_k(w, e) = \{(v, f) : wR_kv,\ \mathcal{M}, v \models pre(f)\}$, which contains (v, e_{nil}) for every $v \in R_k(w)$ because $pre(e_{nil}) = \top$, and contains (w, e), which gives (d). No event changes $task$. (a) If $K_k task$ at w, every $v \in R_k(w)$ has $task$, and so does every pair; otherwise some $v \in R_k(w)$ has $\neg task$, and so does (v, e_{nil}). (b) If $\neg K_k K_j task$ at w, there are $v \in R_k(w)$ and $u \in R_j(v)$ with $\neg task$. Whatever j's class at v, $Q_{j,v}$ maps e_{nil} to a set containing e_{nil}, so $(u, e_{nil}) \in R'_j(v, e_{nil})$ and $(v, e_{nil}) \models \neg K_j task$. (c) is (a) for $busy_j$ and $\neg busy_j$, which no call changes. □

Proposition 2 is the paper's "disconnected robots' knowledge is not impacted", read with Unsure observers: what a robot out of contact knows is what it knew, and it now considers possible both that the call happened and that it did not.

Proposition 3 (Absence is informative; ruleset 1)**.** If $\mathcal{M}, w \models \mathit{pre}(e_{abs})$, then $\mathcal{M} \otimes \mathcal{E}, (w, e_{abs}) \models K_i\, \mathit{busy}_j$, and, by Proposition 2(c), no robot out of contact comes to know whether j is busy.

Proof. i is Fully, so its pairs are (v, e_{abs}) with $\mathcal{M}, v \models \mathit{pre}(e_{abs}) \ni \mathit{busy}_j$. □

The paper's first ruleset turns this knowledge into exclusion. The model keeps it as knowledge, which the precondition of await requires: a robot waits at a particle only for a teammate it knows to be busy.

Proposition 4 (Empathy)**.** Let $j \in B$ and $i \neq j$, and suppose $b \geq 2$ or $i \notin B$. Then $\mathcal{M}_0(I) \models K_j \neg K w_i\, \mathit{busy}_j$.

Proof. Take any $w_{S,t}$ and any $v = w_{S',t'} \in R_j(w_{S,t})$. Robot i cannot distinguish v from a world whose busy set differs from S' only in j, if that world exists: when $i \notin B$, $w_{\{j\},t'}$ and $w_{\emptyset,t'}$ are both i-accessible from v; when $i \in B$ and $i \in S'$, $w_{\{i,j\},t'}$ (which exists since $b \geq 2$) and $w_{\{i\},t'}$ are; when $i \in B \setminus S'$, $w_{\{j\},t'}$ and $w_{\emptyset,t'}$ are. In each case i considers both busy_j and its negation possible at v. □

With $b = 1$ and i busy, i knows that j is not: the bound tells it, as the fault budget does in the heterogeneous paper [2]. The empathy of the paper, "robot j knows what the others believe of it", is $K_j \neg K w_i\, \mathit{busy}_j$: j knows that nobody can rule out that it is at its particle, and so that its particle is where it will be sought.

5.2 The literal reading of eq. (18)

Proposition 5 (Oblivious observers)**.** Let every robot out of contact be oblivious. (a) After any designated event of $\mathsf{call}(i, j)$ at w, the actual pair (w, e) is not among those any bystander k considers possible; under eq. (17), which takes knowledge and belief to coincide, k then knows something false: $K_k \neg K_j\, \mathit{task}$ while $K_j\, \mathit{task}$. (b) If after a met call $i{:}j$ the robot j calls a bystander k of that call, or i waits at the particle of a robot j that was oblivious of i's absent call there, the robot called, or waited for, considers no world possible.

Proof. (a) For an oblivious k, $R'_k(w, e) = \{(v, e_{nil}) : w R_k v\}$, which does not contain (w, e) for $e \neq e_{nil}$. At every (v, e_{nil}), j's pairs are (u, e_{nil}) with $u \in R_j(v)$, among them a world with $\neg \mathit{task}$ when $j \notin K$; so $K_k \neg K_j\, \mathit{task}$, while $K_j\, \mathit{task}$ holds at (w, e_{met}) by Proposition 1. (b) The second call's met event requires $K_j\, \mathit{task}$, which fails at every pair k considers possible, by (a); k is Fully at the met event, so its pairs in the second product are the pairs it considered possible that satisfy that precondition: none. The await event requires by_{ij}, made true by the absent event of which j was oblivious, so false at every pair j considers possible. □

Corollary 1. Under oblivious observers no canonical instance has a policy in which every robot keeps consistent beliefs; Aletheia proves it ("no solution exists") for all four. Three mechanisms lead there. Two are those of Proposition 5(b), and concern robots genuinely out of contact: news that reaches a robot through another (`sim_case1`, five robots whose neighbour relation forces a cascade) and a wait for a busy robot (`hardware_case2`). The third is the arrival that does the task, whose precondition says that the others have arrived: the robot making it was oblivious of their arrivals and believes them not to have happened (all four instances, and any whose task needs two robots or more). The third rests on this model's treatment of arrivals, which robots already at the site do not observe; a model in which they saw each other arrive would remove it, and not the first two. With Unsure observers all four are solved and every robot keeps the actual world at every step (`tools/oblivious.py`).

The heterogeneous paper [2] has the same rule and the same consequence, reported with the same tools in the companion report.

5.3 Joint policies

Definition 6 (Implementable policy)**.** The view of robot a at a step of the branch a designated world takes is the sequence of (action, event) pairs of the earlier steps at which a was Fully. A policy is implementable when, for every robot a and any two designated worlds, wherever a's views coincide and a acts next, it does the same action.

This is the paper's condition that indistinguishable epistemic states map to the same action, and the condition of implicit coordination [6]. Aletheia's `--subjective auto` branches every action on what its actor observes; `tools/check_policy.py` checks the definition directly. Every policy of the four canonical instances is implementable (2 092 checks of the update rules and the goal pass with it).

5.4 Solvability

Theorem 1. Let I be an instance whose neighbour relation is connected, and assume (**A**) that a busy robot comes back to its particle and stays there until it is called. Then I has a solution: a policy, implementable, that reaches the goal from every designated world of $\mathcal{M}_0(I)$, of depth at most $(|\mathcal{A}| - 1) + b + k$.

Proof. Fix a spanning tree T of N rooted at f, and let U be the robots that must learn of the task: those on the paths of T from f to $k-1$ robots outside K in Case II (or K suffices), all of $\mathcal{A}$ in Case I. Visit $U \setminus K$ in an order in which every robot comes after its parent in T, and for each j let its parent i call it; on the absent outcome, i awaits j. By induction on the order, the caller knows *task* when it calls (Proposition 1 for robots told before, the initial model for K), is free (each call or await leaves it free), and $near_{ij}$ holds by construction; after an absent outcome $K_i\, busy_j$ holds (Proposition 3), so the await is applicable, and under (A) it is the event that occurs, after which j knows *task* by the same argument as Proposition 1 (j is Fully and the event requires *task*). Then k robots that know *task* arrive, the last by do-task. There are $|U \setminus K| \le |\mathcal{A}| - 1$ calls, at most b awaits and k arrivals. The branching is on call outcomes, which only the caller observes apart from the robot called; the actions after a branch are the caller's, or those of robots that observed the outcome or learned it in a later call, so the policy is implementable. □

Proposition 6 (Least depth)**.** Let $K = \{f\}$ and $B = \mathcal{A} \setminus \{f\}$. Every solution has, in the worst designated world, at least $(|\mathcal{A}| - 1) + k$ actions in Case I, and at least $(k-1) + \min(b, k-1) + k$ in Case II; when the task needs the whole team, at least $(|\mathcal{A}| - 1) + b + k$.

Proof. A met call or an await tells exactly one robot of the task, and an absent call tells nobody. A robot arrives at the site by an action of its own, after it knows of the task. In Case I all $|\mathcal{A}| - 1$ others must be told, and k must arrive. In Case II at least $k-1$ others must be told and k must arrive; follow the branch of a solution in the world in which the first $\min(b, k-1)$ robots it calls are busy, which is designated because they are in B and are at most b: each of those calls tells nobody, so at least $\min(b, k-1)$ actions beyond the $k-1$ that tell are needed. When the task needs the whole team every robot must be told, and in the world where the first b robots called are busy each needs an action beyond its call. □

In all 159 instances of the scaling study (Figure 2), all with $K = \{f\}$ and $B = \mathcal{A} \setminus \{f\}$, the policies Aletheia returns attain these bounds exactly: the 54 of Case I, the 39 of Case II whose task needs the whole team, and the 66 others. They are of least worst-case depth, and the bound of Theorem 1 is met with equality wherever the whole team must be told.

Assumption (A) is the paper's own empathy: a robot away on other work goes back to where the others believe it to be. The model states it as an event with a single outcome, so a planner that relies on it relies on that assumption, and it does not hold when two policies run at once.

Proposition 7 (Wait-for cycles)**.** Without (A), two policies can wait for each other forever: if a robot of the first waits at the particle of a robot of the second, and that robot, whose policy needs the first robot, waits at the first robot's particle, neither await occurs, since each robot returns to its particle only when its own policy ends.

Before the conventions below were added, the kinematic study produced exactly this with the paper's own task mix, a two-robot and a three-robot task with three robots (Section 6): the finder of the three-robot task sent a teammate to tell the third robot, the finder of the two-robot task found that teammate busy and waited at its particle, and the teammate, finding the finder of the two-robot task away, waited at the finder's. The paper's exclusion never waits, so it cannot deadlock; it is instead incomplete: with $k = |\mathcal{A}|$ and one robot busy when sought, the task cannot be allocated until the excluded robot is met again, which nothing in the rules arranges. The implementation resolves conflicts by two conventions both robots can apply on what they know: when members of two policies are in contact they share both tasks and the task found first keeps its robots; and a robot waits at a particle for at most a bounded time, after which every member of its policy goes back to its particle and the finder plans again after a random back-off. No robot then waits without bound; the mission still depends on the back-offs separating the two finders.

5.5 The second ruleset rests on belief

Proposition 8. Out of contact with j, robot i never comes to know that j knows of a task: if $\mathcal{M}, w \not\models K_i K_j \mathit{task}$, the same holds after any event that i does not observe fully.

Proof. Proposition 2(b), with $k = i$. □

The second ruleset acts on "j's particle saw the task before I did", which yields at most $B_i K_j \mathit{present}(\tau)$ under the hypothesis that j was at its particle. When j was not, the task is left to a robot that does not know of it, until the two are in contact. The procedure applied the rule 5 times in the study; the epistemic formulation does not apply it.

5.6 When exclusion happens

Proposition 9. If every robot not busy stays within ε of its particle and the radio range r_c exceeds ε plus the distance at which a seeker calls a particle empty, a robot sought at its particle is found unless it is busy.

Proof. The seeker calls the particle empty only when it is within that distance of the particle with nobody in range; a robot not busy is within ε of it, so within r_c of the seeker. □

In the paper's setting ($r_c = 10$ m) a robot that is not busy is found where it is believed to be unless it has fallen ten metres behind its particle. The first ruleset fired 24 times in 180 runs of the procedure, and left 3 allocations short of robots for a while.

5.7 Size and planning cost

The ground task has $|\mathcal{A}|(|\mathcal{A}| - 1)$ actions of each of call, await and leave, and $8|\mathcal{A}|$ arrivals of each kind: 296 for eight robots. Its initial model has the size of Lemma 1. The paper counts $(N_a(N_a + N_t))!$ candidate solutions of allocation with gossip; the planner does not enumerate them. With the replanning strategy, 320 policies were planned in the study, all solved, in 0.032 s on average and 0.31 s at most. Figure 2 gives the worst time by team size. For teams of up to four AO*, which returns policies of least depth, finds the same depths, and Proposition 6 shows that every policy found is of least worst-case depth.

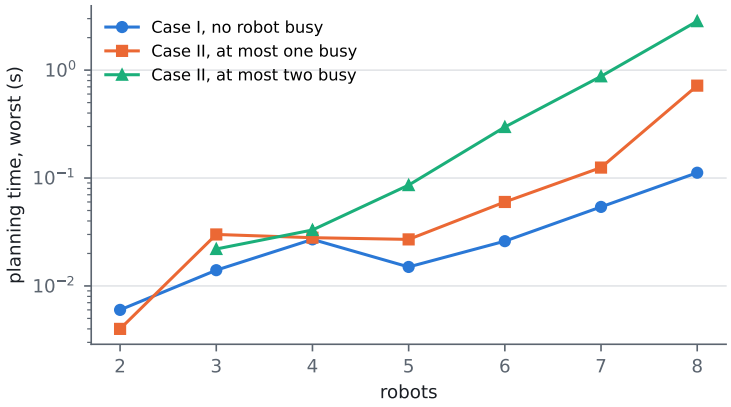


Figure 2: Worst planning time over three random geometries, by team size, for each need $k \in \{2, 3, N\}$ pooled: Aletheia with `--strategy replan --subjective auto --consistent-beliefs`.

6 The paper's simulation study

Setting. Fifteen areas (seeds 1 to 15) with 10 to 20 obstacles of 1 m, the same for every team size, case and formulation; robots of 3 m/s with a 5 m range sensor and Gaussian process noise of 0.2 m/s; particles of 2.7 m/s, so that a robot that detours can catch its particle; a 10 m radio. A mission ends when every task is done and every robot is home, and in Cases II and III when the whole area has been seen. Case III, not in the paper's simulations, has tasks needing more robots in all than the team has, as in its laboratory experiment of Fig. 11: a one-robot and a two-robot task for two robots, a two-robot and a three-robot task for three, and three-robot tasks beyond.

	Case I		Case II	
	paper	here	paper	here
procedure minus ideal, mean (s)	11	7.4	16	8.1
flock minus procedure, mean (s)	13	0.1	35	34.4
standard deviation: ideal / procedure / flock (s)	14 / 11 / 48	16 / 16 / 27		7 / 12 / 9

Table 2: The paper's summary statistics and the recreation's, averaged over the four team sizes.

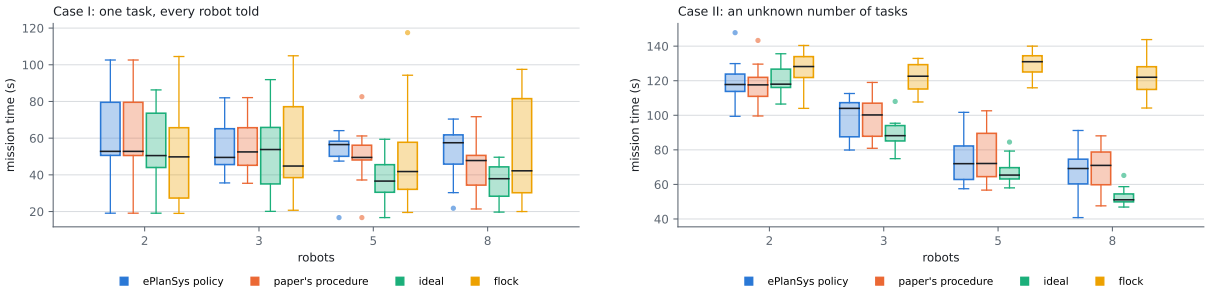


Figure 3: Mission time by team size, fifteen areas per box: the paper's Figures 6 (Case I) and 8 (Case II), recreated, with the epistemic formulation as a fourth box.

The paper's ordering reappears. In Case II the procedure lies between the ideal method and the flock at every team size and approaches the ideal method as the team grows, while the flock does not improve with the team; in Case I the flock is better than the procedure with two robots (mean 53.1 against 62.7 s) and worse with eight (53.1 against 44.6 s), though with eight its median is the lower (42.2 against 47.8 s): its spread is the largest, a standard deviation of 27 s against 16, which is the paper's own observation about the flock. The two gaps of Case II are 8.1 s and

34.4 s against the paper's 16 and 35. Absolute times are higher than the paper's (the ideal method averages 46 s in Case I and 82 s in Case II over the four team sizes, where the paper's figures show about 32 and 40 s): the frontier coverage here leaves more ground to come back for than the paper's MATLAB code, which is not published.

The epistemic formulation. Figure 4 gives, area by area, the epistemic formulation's mission time minus the procedure's. Over the 12 cells of case and team size, the median difference lies between −0.9 and +6.5 s. In Cases II and III the two are at parity: the mean difference per cell lies between −4.9 and +5.2 s, with 47 areas where the policy is faster by more than half a second, 22 ties and 51 where it is slower. In Case I, where every robot must be told, the policy is slower as the team grows (median +1.4 s with five robots and +6.5 s with eight): Aletheia minimises the number of steps, and its cascades follow the neighbour relation, where the paper's gossip auction weighs travel times. Every one of the 320 policies the study asked for was found, in 0.032 s on average and 0.31 s at most.

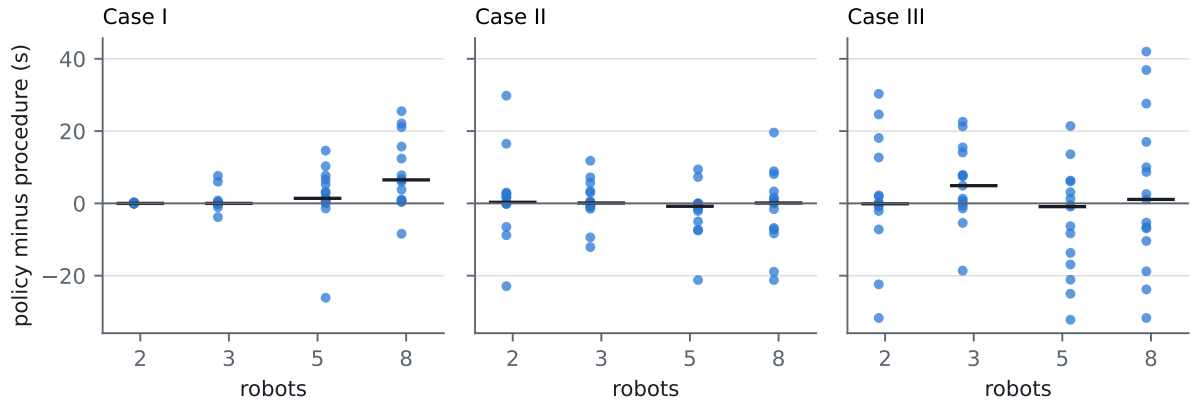


Figure 4: Epistemic formulation minus the procedure, per area; the bar is the median.

7 The laboratory experiments in Gazebo

Arena. The paper's 4×5.5 m laboratory at twice its size: an 8×11 m space with five crates of 0.5 m, unknown to the robots, a metre or more from every wall, and the base, a patch of 2×1.2 m, at the south wall; a robot is home once it is within 1 m of the base's centre, and goes on to its own place there; sensing and radio ranges of 2 m; three Burgers whose particles move at 0.13 m/s. Every length of the coverage is the study's scaled by 2/5. Case I is the paper's Fig. 10: one task, for two robots or all three. Case II is its Fig. 11: a task for two robots, which takes 40 s at the site, and a task for three; the seed's placements are screened, in the kinematic simulator under the procedure, until the three-robot task is found while the two-robot task is being done.

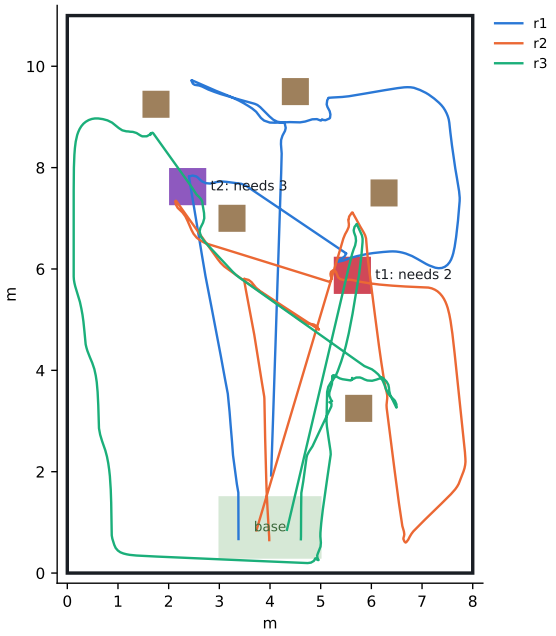


Figure 5: The arena of Case II, seed 1, under the epistemic formulation: crates, base, the two tasks with the number of robots each needs, and the robots' tracks.

ePlanSys. Under the epistemic formulation, the finder's instance is written as an EPDDL problem and grounded by plank; ePlanSys's epistemic state loads it (`LoadTask`); its planner plans it with Aletheia through `plansys2/AletheiaPlanSolver`; its epistemic executor runs the policy with parallel dispatch, as `schedule_policy` arranges it, over a vocabulary in which every action reads and writes the readiness of the robots it names, and an arrival reads the count the arrival before it writes, so that arrivals are dispatched in the order the epistemic state requires. Each action reaches the mission through a relay and is answered with the event that occurred. When all three robots come into contact the common belief is reset and the allocation is made with everything known, under every formulation; a policy still running is ended there and its task finished under that allocation. One executor runs one policy at a time, so a task found while a policy runs waits for it.

Results. 96 of 96 runs ended with every task done and every robot home. Against the procedure, the epistemic formulation's mission time differs by a median of $+0.6$ s in Case I and -0.4 s in Case II (faster, tied and slower in 3, 3 and 6 seeds, and in 6, 1 and 5); the mean differences, $+12.6$ and -17.8 s, are carried by a few seeds each, and a sign test separates neither case ($p = 0.51$ and 1.00). ePlanSys planned 23 policies, each loaded, planned and handed to the executor in 0.40 s at most, and every call was answered with the event that occurred. Of them, 6 ran to their end and 16 were ended when all three robots came into contact, and the allocation made with everything known took their remaining steps: in 11 of them the policy had already told and sent all three robots, which met on their way to the site, and in the others contact came during a call. One policy failed, in Case I with seed 10: a robot that was not busy had fallen 2.5 m behind its particle among the crates, so the call found nobody where the finder's model allowed no absence (the hypothesis of Proposition 9 failed). ePlanSys's epistemic state refused the outcome, the finder planned again with the robot it had told left out, and the new policy did the task. In Case II the flock, which never leaves radio range, is the slowest by 48 s in median, since the whole arena must be seen; in Case I, where the one task is near, the medians run the flock 120, the ideal method 121, the procedure 144, the ePlanSys policy 148 s.

case	formulation	ended	mean	median	max	tasks done by
I	epistemic	12 / 12	149.7	147.8	261.4	113.3
I	proposed	12 / 12	137.1	144.0	218.5	101.6
I	ideal	12 / 12	120.2	121.2	180.1	84.0
I	flock	12 / 12	131.3	120.4	298.0	94.7
II	epistemic	12 / 12	301.0	309.9	391.4	160.8
II	proposed	12 / 12	318.8	324.6	450.3	163.8
II	ideal	12 / 12	294.1	279.9	389.4	135.2
II	flock	12 / 12	383.4	372.4	447.6	174.2

Table 3: Gazebo, 12 seeds per case: mission time (s).

8 Implementation defects

1. *Arrivals out of order.* With a vocabulary in which an arrival read and wrote only its robot's readiness, ePlanSys's scheduler moved the arrival that does the task above the other arrivals, since nothing said it depended on them, and the group that held it waited forever. With the task reading every count below its need, the arrivals before it could still change places: in one Gazebo run two robots knew of the task when it was found, and the second arrival was dispatched with a call, before the first arrival. ePlanSys's epistemic state refused the update, since the event of the second arrival requires the count the first sets, and the policy failed twice. Each arrival now reads the count of the arrival before it, and the scheduler check of `validate.sh` includes this situation.

2. *A robot on its way home taken for busy.* A robot heads home when it has nothing left to do: in Case I once it knows of the task, and in either case once the area has been covered. A robot that had turned for home was not counted as free, so a call that reached it, in range and idle, came back absent, an outcome the finder's instance did not allow since nobody could be busy, and the policy failed. A robot heading home, or at home, is free.

3. *A teammate told and gone home.* In Case I a robot that knows of the task and has nothing to do goes home. When a policy failed and its finder planned again, the new instance counted the robots told under the old one as robots at their particles still to be told, and its calls went to particles they had left. A robot now keeps, for each task, the robots it knows to know of it: robots in contact know of the same tasks and know that they do, and a call that finds its robot adds both. In Case I the finder leaves out of its instance the robots so known that are out of contact, unless the task needs them.

4. *Robots held up at the base.* A robot counted as home only near its own place at the base, and robots coming home in single file stopped one another short of their places: one run never ended, and in others a robot waited behind another for up to a minute, which went into the mission time. A robot is now home once it is within a metre of the base's centre, and goes on to its place from there.

5. *Groups that wait for a drive.* ePlanSys starts a group of independent actions together and the next group when all have finished; an arrival answered when the robot reached the site held the next calls for the length of a drive. An arrival is answered when the robot sets out, and the task waits at the site for everyone, as in the companion demonstration [2].

6. *Relay nodes that shared a name.* A launch `name=` renames every node of a process; the relay process, which hosts one action client per action, lost all but one. The relays are launched without it.

7. *A common belief reset every second.* Reset at every instant all robots were in contact, the particles began again facing north, and three robots in contact never left one another; the paper

resets it when new knowledge is shared. The reset now keeps each particle's heading and goal, and happens when tasks, allocations or obstacles are shared, or at the latest every 10 s.

8. *Coverage the belief claimed through an unseen obstacle.* A particle sees through obstacles nobody has seen, and could call covered a pocket its robot could not see, so that the team went home, found the pocket open, and went out again, for minutes. A cell at the edge of what the robots have really seen counts as seen in the belief only from beside it.

9. *Two policies waiting for each other* (Proposition 7).

The first four were found by the Gazebo runs. The Gazebo runs reported here were all made after the four corrections, and the kinematic study's epistemic runs after the first three (the fourth does not apply there, where robots have no size).

9 Limitations

The kinematic simulator is not the paper's code: its coverage is slower, its cases are drawn by its own rules, and the procedure includes a strengthening of the first ruleset. The planner minimises steps in the worst case, not time: it prefers waiting for a busy robot (one step) to calling another, and its cascades follow the neighbour relation, not travel times, which is the likeliest reason the epistemic formulation is slower in Case I with eight robots. Gazebo runs differ in their timing from one run to the next; with twelve seeds per case the mission times set the flock apart in Case II, but do not separate the procedure from the epistemic formulation. In Gazebo one ePlanSys executor runs one policy at a time, so a task found while a policy runs waits for it. The Gazebo arena is the paper's laboratory scaled, not its laboratory; its sensing is simulated over the arena's map, with the laser used only to stop short of obstacles. The analysis is of the planning task at a task found; the coverage, the allocation of several tasks at once and the conventions of Proposition 7 are outside the model.

References

[1] L. Bramblett and N. Bezzo. Epistemic planning for multi-robot systems in communication-restricted environments. *Frontiers in Robotics and AI* 10:1149439, 2023.

[2] L. Bramblett and N. Bezzo. Epistemic planning for heterogeneous robotic systems. arXiv:2308.00579, 2023.

[3] H. van Ditmarsch, W. van der Hoek and B. Kooi. *Dynamic Epistemic Logic*. Springer, 2007.

[4] A. Baltag, L. S. Moss and S. Solecki. The logic of public announcements, common knowledge, and private suspicions. *Proceedings of TARK VII*, 43–56, 1998.

[5] T. Bolander and M. B. Andersen. Epistemic planning for single- and multi-agent systems. *Journal of Applied Non-Classical Logics* 21(1):9–34, 2011.

[6] T. Engesser, T. Bolander, R. Mattmüller and B. Nebel. Cooperative epistemic multi-agent planning for implicit coordination. *Proceedings of the Ninth Workshop on Methods for Modalities*, EPTCS 243, 75–90, 2017.